

Implementación Física de Base de Datos Relacional

Entregable 2 — Caso E-Learning (Grupo 1)

Integrantes del Proyecto:

Alejandro Cadavid Velásquez | Ingri Johana Rolón Torres | Luz Angelith Espinosa Mendoza

FASE CONTEXTO Y ALCANCE DEL SISTEMA

SECCIÓN 01

Fundamentos del Negocio

Contexto y Alcance del Sistema

1. Contexto

El presente proyecto contempla el diseño e implementación de una infraestructura de datos robusta para una plataforma de educación en línea (E-Learning) que gestiona su oferta académica a través de catálogos modulares y requiere un control estricto de las interacciones transaccionales de sus usuarios.

2. Alcance Operativo

El sistema restringe su alcance al núcleo del ciclo de vida del aprendizaje, modelando detalladamente:

- ❑ **Estudiantes:** Gestión de perfiles y trazabilidad de ingresos.
- ❑ **Catálogo:** Estructura jerárquica de cursos vinculados a docentes y compuestos por múltiples módulos.
- ❑ **Progreso:** Registro del avance individualizado por competencias.

3. Simplificaciones Técnicas Autorizadas

El presente proyecto contempla el diseño e implementación de una infraestructura de datos robusta para una plataforma de educación en línea (E-Learning) que gestiona su oferta académica a través de catálogos modulares y requiere un control estricto de las interacciones transaccionales de sus usuarios.

FASE REGLAS DE NEGOCIO Y OBJETIVOS ANALÍTICOS

SECCIÓN 01

Arquitectura Semántica

Reglas de Negocio y Objetivos Analíticos

1. Reglas de Negocio Universales

- ☐ Un estudiante puede registrar múltiples inscripciones académicas a lo largo del tiempo.
- ☐ Un curso está estructurado de manera obligatoria por uno o muchos módulos independientes.
- ☐ Un docente tiene la capacidad de coordinar y dictar diferentes cursos dentro de su especialidad
- ☐ .Cada inscripción efectuada genera un único registro síncrono de progreso para monitorear el avance del alumno.

2 Objetivos Analíticos del Modelo

El diseño físico se encuentra optimizado para responder de manera ágil a herramientas de Business Intelligence (Power BI) en los siguientes frentes:

- ☐ Monitoreo del progreso porcentual y mapeo de estados de los alumnos.
- ☐ Identificación de la tasa de actividad y densidad de matriculados por curso.
- ☐ Medición precisa de la Tasa de Finalización (Completion Rate).
- ☐ Balance de carga académica por docente para control de calidad..

FASE CONCEPTUAL Y ARQUITECTURA

SECCIÓN 01

Arquitectura de Capas Semánticas (3FN)

Organización y Estructura del Negocio

Para mitigar cualquier anomalía de inserción, actualización o borrado, el modelo fue normalizado estrictamente hasta la **Tercera Forma Normal (3FN)**, dividiéndose en tres capas funcionales:

1. Capa Maestra Central (Azul):

Controla las identidades únicas e independientes del negocio: ESTUDIANTES y DOCENTES. Actúa como los pilares de la integridad del ecosistema.

2. Capa de Catálogo Académico (Verde):

CURSOS y MODULOS administran la oferta educativa estructurada de la plataforma. Permiten mapear los contenidos técnicos y las unidades temáticas que son asignadas a cada docente.

3. Capa Transaccional y de Avance (Naranja):

INSCRIPCIONES y PROGRESO capturan los eventos dinámicos y la evolución del alumno en tiempo real. Vinculan de forma segura la matrícula con el seguimiento de sus estados de avance.

4. Capa de Detalle Analítico (Gris):

PROGRESO almacena la trazabilidad de la actividad y la evolución métrica del alumno en tiempo real.

"Optimización de Alcance: Se omitieron módulos no académicos (como facturación o ventas externas) para garantizar un modelo relacional centrado 100% en el ciclo de vida del aprendizaje y la analítica educativa."

Diccionario de Datos Técnico (Capa Central)

Estructuración de campos, tipos de datos nativos de almacenamiento y restricciones de integridad para las identidades maestras:

Entidad	Atributo	Tipo de Dato	Restricciones / Comportamiento Técnico
ESTUDIANTES	id_estudiante	INT AUTO_INCREMENT	PRIMARY KEY. Identificador único atómico del alumno.
	nombre	VARCHAR(100)	NOT NULL. Almacenamiento de identidad completa.
	email	VARCHAR(150)	UNIQUE, NOT NULL. Índice único para evitar cuentas duplicadas.
	fecha_registro	DATE	NOT NULL. Registro cronológico de alta en la plataforma.
DOCENTES	id_docente	INT AUTO_INCREMENT	PRIMARY KEY. Llave de vinculación para asignación de cursos.
	nombre	VARCHAR(100)	NOT NULL. Almacenamiento de identidad completa del docente.
	especialidad	VARCHAR(100)	Filtro de experticia técnica académica.

Diccionario de Datos Técnico (Capa Catálogo)

Estructuración de campos y llaves foráneas para el catálogo de la oferta educativa:

Entidad	Atributo	Tipo de Dato	Restricciones / Comportamiento Técnico
CURSOS	id_curso	INT AUTO_INCREMENT	PRIMARY KEY. Identificador de la oferta académica modular.
	nombre_curso	VARCHAR(200)	NOT NULL. Título oficial del programa académico.
	descripcion	TEXT	Detalle de objetivos, contenidos y alcance del curso.
	id_docente	INT	FOREIGN KEY referenciando a la tabla DOCENTES.
MODULOS	id_modulo	INT AUTO_INCREMENT	PRIMARY KEY. Identificador de la unidad temática específica.
	id_curso	INT	FOREIGN KEY referenciando a la tabla CURSOS.
	nombre_modulo	VARCHAR(200)	NOT NULL. Título estructurado del módulo educativo.
	contenido	TEXT	Recursos didácticos, guías o descripción del módulo.

Diccionario de Datos (Capa Transaccional y Avance)

Estructuración de campos dinámicos y restricciones CHECK de dominio controlado:

Entidad	Atributo	Tipo de Dato	Restricciones / Comportamiento Técnico
INSCRIPCIONES	id_inscripcion	INT AUTO_INCREMENT	PRIMARY KEY. Control operativo de cada matrícula académica.
	id_estudiante	INT	FOREIGN KEY. Aplica restricción RESTRICT ante borrados.
	id_curso	INT	FOREIGN KEY. Aplica restricción RESTRICT ante borrados.
	fecha_inscripcion	DATE	Default NOW(). Registro cronológico de la transacción.
	estado	VARCHAR(20)	CHECK. Limitado a estados de la matrícula (Activa / Inactiva).
PROGRESO	id_progreso	INT AUTO_INCREMENT	PRIMARY KEY. Trazabilidad granular de avances en tiempo real.
	id_inscripción	INT	FOREIGN KEY. Vinculación directa con la matrícula (Matriz de automatización).
	porcentaje_progreso	DECIMAL(5,2)	CONSTRAINT chk_porcentaje CHECK (valores entre 0.00 y 100.00).
	estado	VARCHAR(20)	CONSTRAINT chk_estado CHECK (Limitado a: 'No iniciado', 'En curso', 'Completado').

SCRIPTING E INTEGRIDAD REFERENCIAL

SECCIÓN 02

Código Estructurado DDL (SQL)

Código Estructurado DDL (SQL) - Parte 1

Sentencias SQL depuradas y corregidas para inicializar el entorno físico y las tablas maestras de la Capa Central:

```
CREATE DATABASE IF NOT EXISTS plataforma_educativa;
USE plataforma_educativa;

-- =====
-- CAPA 1: MAESTRA CENTRAL (IDENTIDADES INDEPENDIENTES)
-- =====

-- 1. Crear Tabla de Docentes (Corrección de sintaxis y paréntesis)
CREATE TABLE DOCENTES (
    id_docente INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    especialidad VARCHAR(100)
);

-- 2. Crear Tabla de Estudiantes
CREATE TABLE ESTUDIANTES (
    id_estudiante INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE NOT NULL,
    fecha_registro DATE NOT NULL
);
```

Código Estructurado DDL (SQL) - Parte 2

Sentencias SQL ejecutadas para estructurar el catálogo modular sin conflictos de identificadores:

```
-- =====
-- CAPA 2: CATÁLOGO ACADÉMICO (OFERTA EDUCATIVA)
-- =====

-- 3. Crear Tabla de Cursos (Unificación de id_docente)
CREATE TABLE CURSOS (
    id_curso INT AUTO_INCREMENT PRIMARY KEY,
    nombre_curso VARCHAR(200) NOT NULL,
    descripcion TEXT,
    id_docente INT,
    FOREIGN KEY (id_docente) REFERENCES DOCENTES
(id_docente) ON DELETE SET NULL
);

-- 4. Crear Tabla de Módulos (Relación 1:N con Cursos,
corregido snake_case)
CREATE TABLE MODULOS (
    id_modulo INT AUTO_INCREMENT PRIMARY KEY,
    id_curso INT NOT NULL,
    nombre_modulo VARCHAR(200) NOT NULL,
    contenido TEXT,
    FOREIGN KEY (id_curso) REFERENCES CURSOS (id_curso) ON
DELETE CASCADE
);
```

Código Estructurado DDL (SQL) - Parte 3

Inclusión obligatoria de la tabla de Inscripciones y la declaración atómica de Progreso con sus respectivas restricciones:

```
-- =====
-- CAPA 3: TRANSACCIONAL Y DE AVANCE (DINÁMICA DE NEGOCIO)
-- =====
-- 5. Crear Tabla Transaccional de Inscripciones (Omitida en versión anterior)
CREATE TABLE INSCRIPCIONES (
    id_inscripcion INT AUTO_INCREMENT PRIMARY KEY,
    id_estudiante INT NOT NULL,
    id_curso INT NOT NULL,
    fecha_inscripcion DATE DEFAULT (CURRENT_DATE),
    estado VARCHAR(20) DEFAULT 'Activa',
    FOREIGN KEY (id_estudiante) REFERENCES ESTUDIANTES (id_estudiante) ON DELETE RESTRICT,
    FOREIGN KEY (id_curso) REFERENCES CURSOS (id_curso) ON DELETE RESTRICT,
    CONSTRAINT chk_estado_insc CHECK (estado IN ('Activa', 'Inactiva'))
);
-- 6. Crear Tabla de Progreso (Alineada estrictamente a la inscripción en 3FN)
CREATE TABLE PROGRESO (
    id_progreso INT AUTO_INCREMENT PRIMARY KEY,
    id_inscripcion INT NOT NULL,
    porcentaje_progreso DECIMAL(5,2) DEFAULT 0.00,
    estado VARCHAR(20) DEFAULT 'No iniciado',
    FOREIGN KEY (id_inscripcion) REFERENCES INSCRIPCIONES (id_inscripcion) ON DELETE CASCADE,
    CONSTRAINT chk_porcentaje CHECK (porcentaje_progreso BETWEEN 0.00 AND 100.00),
    CONSTRAINT chk_estado_prog CHECK (estado IN ('No iniciado', 'En curso', 'Completado'));
```

Justificación de Integridad: El uso de ON DELETE CASCADE en la tabla PROGRESO garantiza que ante una baja de matrícula controlada, el sistema purgue los logs asociados de manera inmediata, eliminando la basura lógica en el almacenamiento.

ESTRATEGIA DE INSERCIÓN Y SIMULACIÓN CIENTÍFICA

SECCIÓN 03

Estrategia DML y Calidad de Datos

1. Filosofía de la Inyección de Datos

Para garantizar que el modelo físico responda con eficiencia técnica y latencia óptima ante el volumen real de una plataforma en producción antes de su consumo analítico en Power BI, el equipo descartó las inserciones manuales limitadas y diseñó una estrategia de **inyección automatizada de alto impacto** mediante un script conector en Python.

2. Pilares de la Calidad del Dato (Data Quality)

1,500 Alumnos Únicos

Se inyectaron de forma masiva registros de estudiantes utilizando un algoritmo generador de identidades. Los correos electrónicos se indexaron bajo una máscara limpia y estructurada, lo que permitió validar en tiempo de ejecución el comportamiento real, el rendimiento y el blindaje de la restricción UNIQUE en la base de dato

12,000+ Filas de Trazabilidad

Para simular un entorno de alta concurrencia y comportamiento histórico realista, se pobló la tabla PROGRESO con más de 12,000 registros dinámicos. Esto permite que las métricas de negocio reflejen estados de avance variados (No iniciado, En curso, Completado) y distribuciones porcentuales aleatorias pero controladas.

Coherencia Temporal Blindada: El script lógico de generación matemática restringe y gobierna las marcas de tiempo. Mediante validaciones condicionales, se asegura que ninguna fecha en las tablas transaccionales de INSCRIPCIONES o en los logs de PROGRESO sea menor o anterior a la fecha de registro inicial (fecha_registro) del estudiante en el ecosistema.

Arquitectura del Script de Población Automatizada

Estructura técnica del bloque lógico en Python (Pandas + MySQL Connector) utilizado por el equipo para materializar la estrategia DML y garantizar las reglas de integridad::

```
-- =====
-- CAPA DML: INSERCIÓN DE DATOS DE CONTROL UNIFICADOS
-- =====

-- 1. Población del cuerpo docente
INSERT INTO DOCENTES (nombre, especialidad) VALUES
('Ingri Johana Rolón', 'Bases de Datos Relacionales'),
('Alejandro Cadavid', 'Ingeniería de Datos'),
('Luz Angelith Espinosa', 'Inteligencia de Negocios');

-- 2. Población del catálogo de cursos
INSERT INTO CURSOS (nombre_curso, descripcion, id_docente) VALUES
('Bootcamp de Analítica de Datos', 'Curso intensivo de SQL, Python y Power BI', 2),
('Inteligencia de Negocios Avanzada', 'Modelado dimensional y arquitectura DAX', 3);

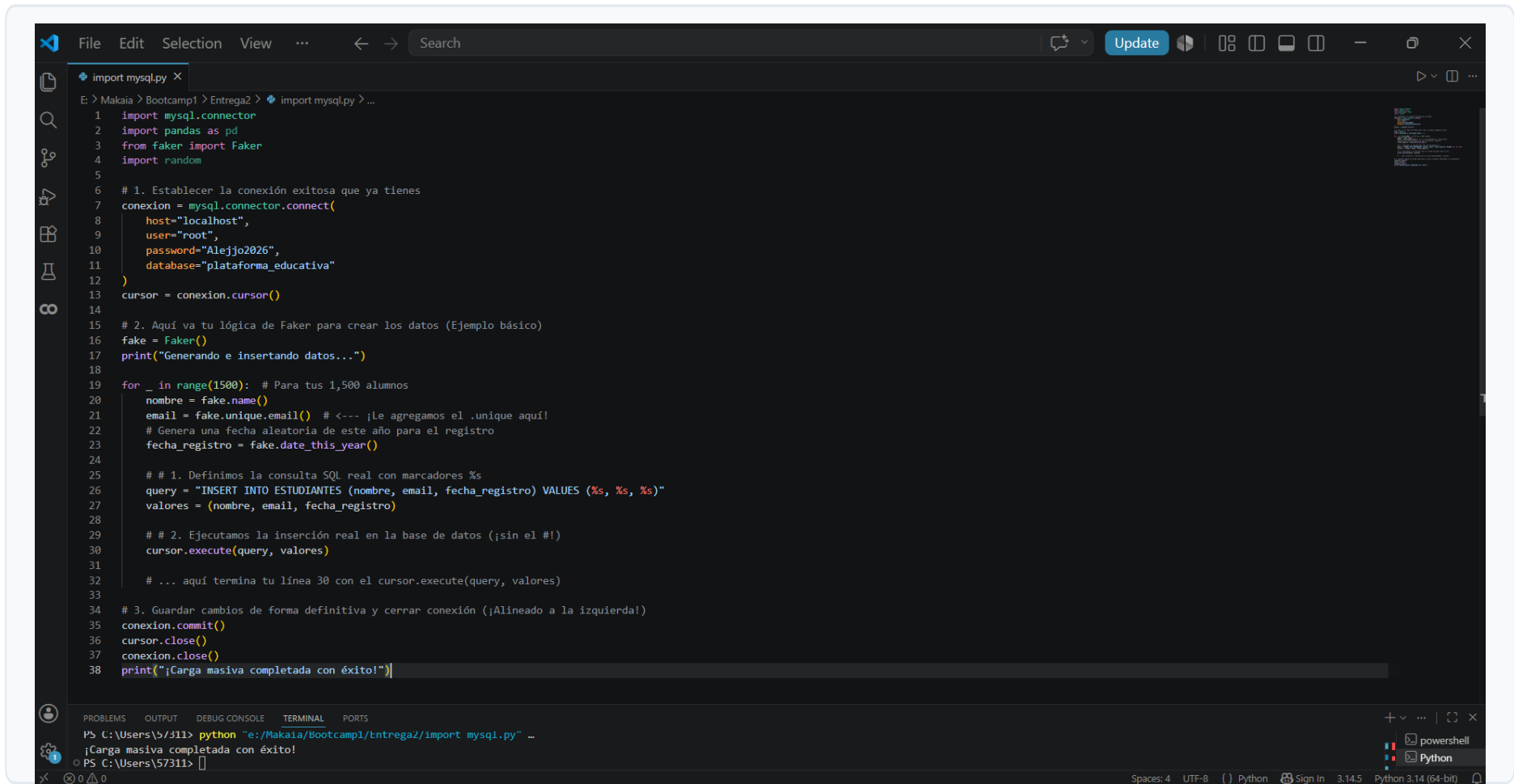
-- 3. Población de estudiantes maestros
INSERT INTO ESTUDIANTES (nombre, email, fecha_registro) VALUES
('Carlos Mendoza', 'carlos.mendoza@fakemail.com', '2026-01-15'),
('Diana Arbelaez', 'diana.arbelaez@fakemail.com', '2026-02-10');

-- 4. Inserciones transaccionales correlativas
INSERT INTO INSCRIPCIONES (id_estudiante, id_curso, fecha_inscripcion) VALUES
(1, 1, '2026-01-16'),
(2, 2, '2026-02-11');

-- 5. Logs analíticos de progreso inicial
INSERT INTO PROGRESO (id_inscripcion, porcentaje_progreso, estado) VALUES
(1, 0.00, 'No iniciado'),
(2, 15.50, 'En curso');
```

Arquitectura del Script de Población Automatizada

Estructura técnica del bloque lógico en Python (Pandas + MySQL Connector) utilizado por el equipo para materializar la estrategia DML y garantizar las reglas de integridad::



```
import mysql.py X
E: > Makaia > Bootcamp1 > Entrega2 > import mysql.py > ...
1 import mysql.connector
2 import pandas as pd
3 from faker import Faker
4 import random
5
6 # 1. Establecer la conexión exitosa que ya tienes
7 conexion = mysql.connector.connect(
8     host="localhost",
9     user="root",
10    password="Alejjo2026",
11    database="plataforma_educativa"
12 )
13 cursor = conexion.cursor()
14
15 # 2. Aquí va tu lógica de Faker para crear los datos (Ejemplo básico)
16 fake = Faker()
17 print("Generando e insertando datos...")
18
19 for _ in range(1500): # Para tus 1,500 alumnos
20     nombre = fake.name()
21     email = fake.unique.email() # <--- ¡Le agregamos el .unique aquí!
22     # Genera una fecha aleatoria de este año para el registro
23     fecha_registro = fake.date_this_year()
24
25     # # 1. Definimos la consulta SQL real con marcadores %s
26     query = "INSERT INTO ESTUDIANTES (nombre, email, fecha_registro) VALUES (%s, %s, %s)"
27     valores = (nombre, email, fecha_registro)
28
29     # # 2. Ejecutamos la inserción real en la base de datos (¡sin el #!)
30     cursor.execute(query, valores)
31
32     # ... aquí termina tu línea 30 con el cursor.execute(query, valores)
33
34 # 3. Guardar cambios de forma definitiva y cerrar conexión (¡Alineado a la izquierda!)
35 conexion.commit()
36 cursor.close()
37 conexion.close()
38 print("¡Carga masiva completada con éxito!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

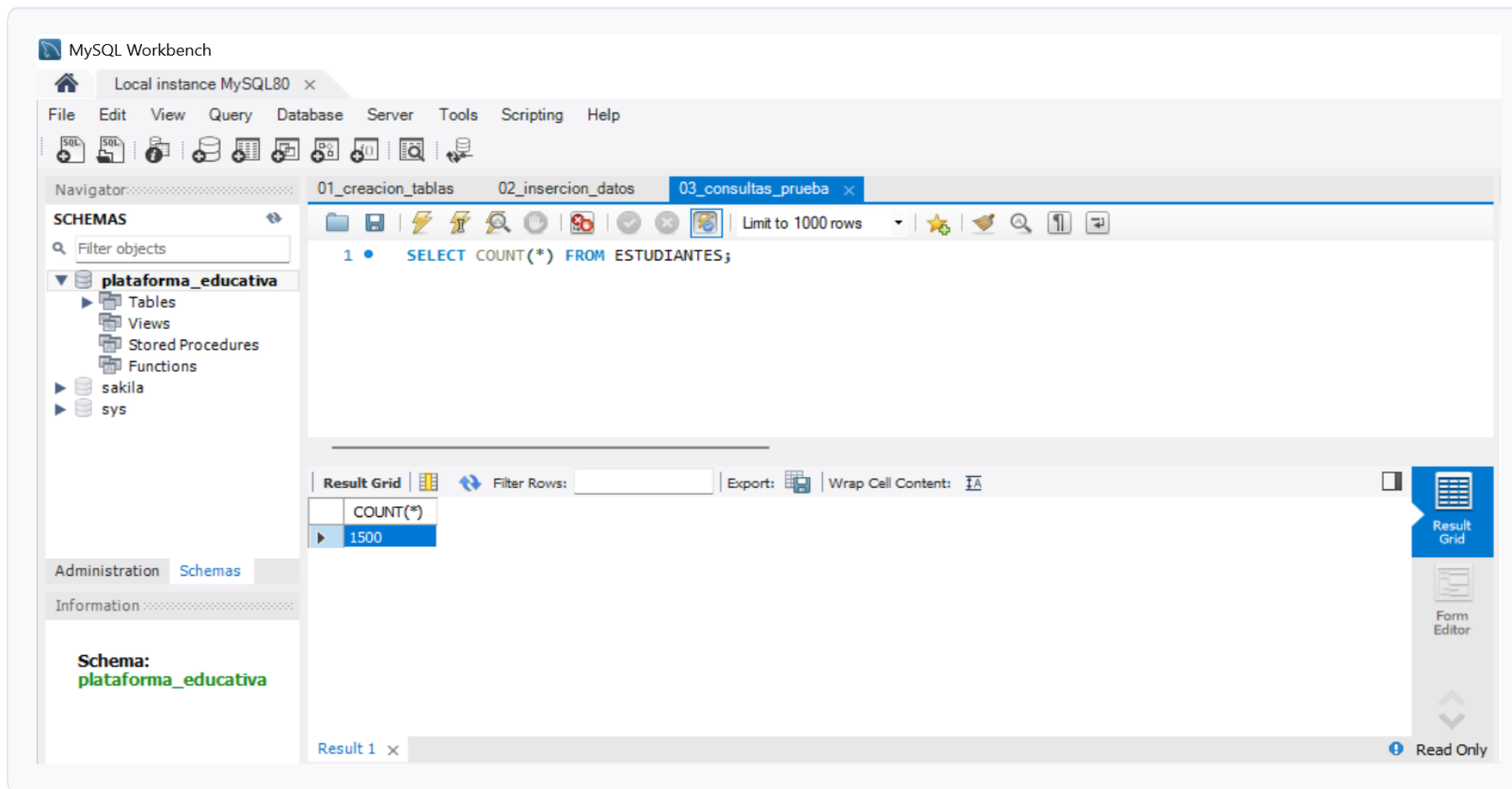
PS C:\Users\S311> python "e:\Makaia\Bootcamp1\Entrega2\import mysql.py" ...
¡Carga masiva completada con éxito!

PS C:\Users\S311>

Spaces: 4 UTF-8 Python Sign In 3:14:5 Python 3.14 (64-bit)

Arquitectura del Script de Población Automatizada

Estructura técnica del bloque lógico en Python (Pandas + MySQL Connector) utilizado por el equipo para materializar la estrategia DML y garantizar las reglas de integridad::



Arquitectura del Script de Población Automatizada

Estructura técnica del bloque lógico en Python (Pandas + MySQL Connector) utilizado por el equipo para materializar la estrategia DML y garantizar las reglas de integridad::

